



**RAJALAKSHMI ENGINEERING COLLEGE,
CHENNAI**

Department of Artificial Intelligence & Data Science

CS23633 – Cloud Computing Laboratory

Experiment 2: Setting up Public Cloud

Aim

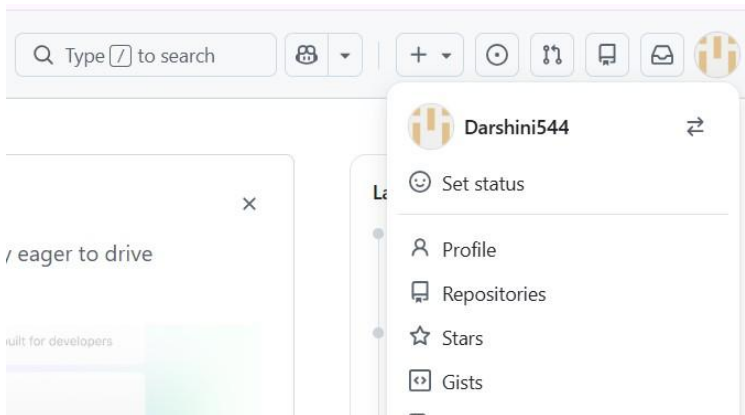
To develop and deploy a simple web application using **Render**, a **Platform as a Service (PaaS)**, and understand how cloud platforms host applications without requiring users to manage servers.

Procedure

Step 1: Sign in GitHub Account

1. Open <https://github.com>
2. Click **Signin**

GitHub stores your application code online.



Step 2: Create a Project Folder

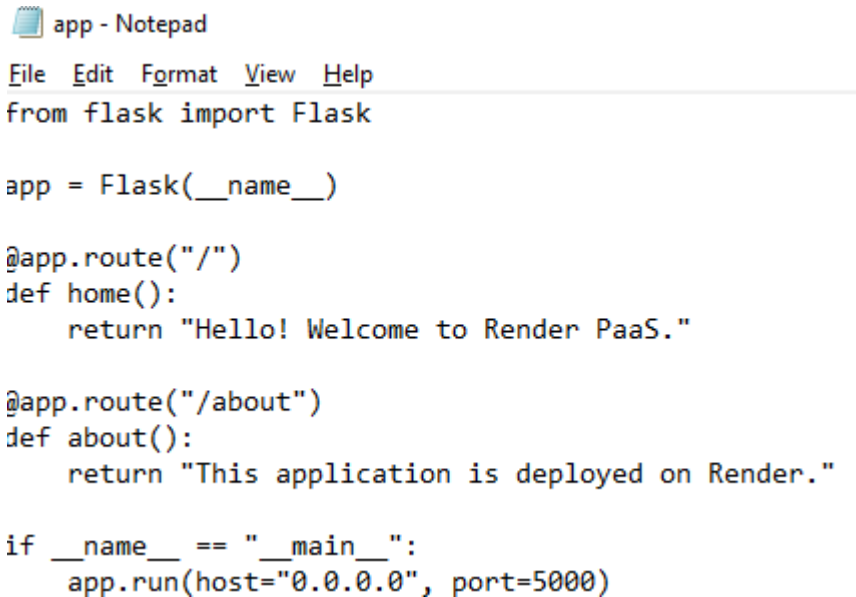
1. Create a folder named RenderHelloApp in your local computer

2. Inside it create two files.

RenderHelloApp

- app.py
- requirements.txt

3. Paste the following content in app.py



```
File Edit Format View Help
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello! Welcome to Render PaaS."

@app.route("/about")
def about():
    return "This application is deployed on Render."

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

4. Paste the following content in requirements.txt



```
File Edit Format View Help
Flask==3.0.3
gunicorn==22.0.0
```

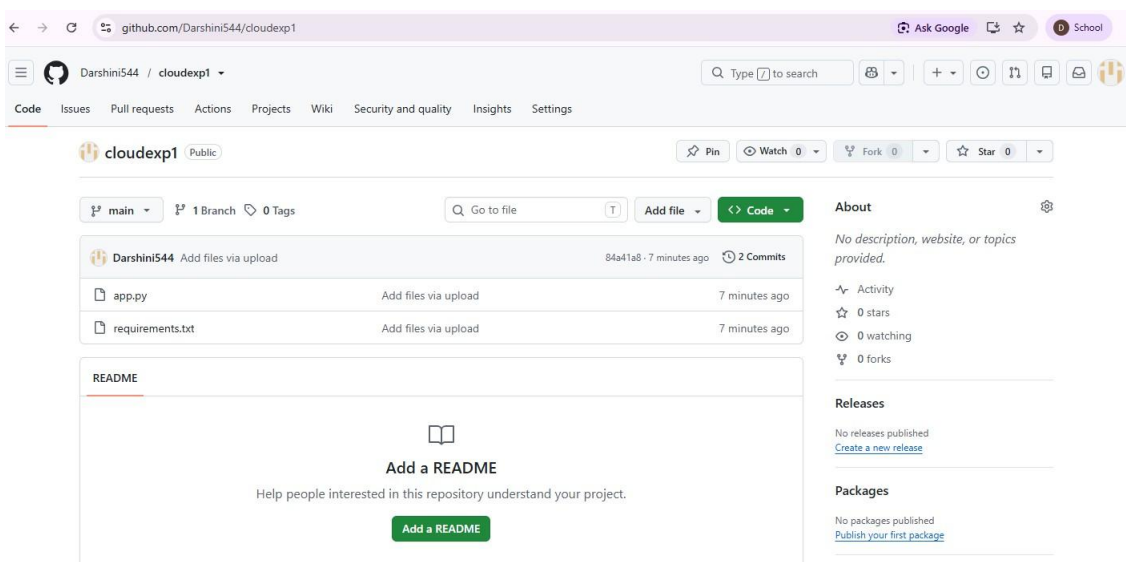
Step 3: Upload the Folder in GitHub

Create a Repository

1. Click **New Repository**.
2. Repository name: cloudexp1
3. Choose **Public**.
4. Click **Create Repository**.(cloudexp1)

Upload Your Files

1. On the new repository page:
2. On this page, there is usually a link that says:
uploading an existing file
3. Click **uploading an existing file**. This opens the upload page where you can drag and drop `app.py` and `requirements.txt`.
4. You will see a page that says:
Drag files here to add them to your repository
5. Click the green **Commit changes** button.



This last step is essential. Until you click **Commit changes**, the files are **not actually saved** in the repository.

Step 4: Create a Render Account

1. Open <https://render.com>
2. Click **Get Started**.
3. After you log in to **Render**, it asks you to choose what type of service you want to create.

For your Flask PaaS experiment, choose:

✓ **Web Service**

This is the correct option for hosting a Python web application.

Steps

1. Log in to Render.
2. On the "Choose a service" page, select **Web Service**.
3. If prompted, connect your **GitHub** account.
4. Select the GitHub repository that contains your Flask application.

5. Configure it as follows:

Setting	Value
Name	renderhelloapp (or any unique name)
Language/Environment	Python
Branch	main
Root Directory	Leave blank
Build Command	<code>pip install -r requirements.txt</code>
Start Command	<code>gunicorn app:app</code>
Instance Type	Free (if available)

6. Click **Deploy Web Service**.

7. After successful deployment you will see the following screen.

The screenshot shows the Render dashboard for a project named 'cloudexp1'. The left sidebar contains navigation options: Environment, cloudexp1, Deploys, Settings, MONITOR (Events, Logs, Metrics), and MANAGE (Environment, Shell, Scaling, Previews, Disk, Changelog, Invite a friend, Contact support). The main content area shows the 'cloudexp1' service with a 'Python 3' environment and a 'Free' instance type. A deployment is in progress, starting on July 17, 2026, at 11:30 AM. The deployment logs show the following steps:

- 11:30:53 AM: Downloading werkzeug-3.1.8-py3-none-any.whl (226 kB)
- 11:30:53 AM: Downloading packaging-26.2-py3-none-any.whl (100 kB)
- 11:30:54 AM: Installing collected packages: packaging, MarkupSafe, itsdangerous, click, blinker, Werkzeug, Jinja2, gunicorn, Flask
- 11:30:54 AM: Successfully installed Flask-3.0.3 Jinja2-3.1.6 MarkupSafe-3.0.3 Werkzeug-3.1.8 blinker-1.9.0 click-8.4.2

Below the logs, a status bar indicates 'July 17, 2026 at 11:30 AM' with a 'Live' status. A link to 'Add files via upload' is provided. The bottom section shows a detailed log of the deployment process, including the command 'gunicorn app:app' and the message 'Your service is live'.

Step 5: Open Your Application

At the top of your screenshot, you can see the URL:

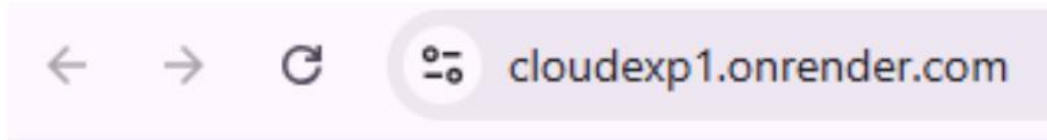
<https://cloudexp1.onrender.com>

Click that link (or copy and paste it into your browser).

Expected Output

If everything is working correctly, you should see:

Hello! Welcome to Render PaaS.



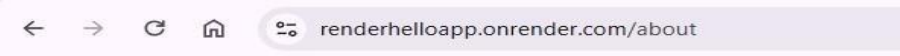
Hello! Welcome to Render PaaS.

Then try visiting:

<https://renderhelloapp.onrender.com/about>

You should see:

This application is deployed on Render.



This application is deployed on Render.

Step 6: Verification

Take screenshots of:

1. **Render Dashboard** (showing "Deploy live") ✓
2. **Home page** (<https://renderhelloapp.onrender.com>)
3. **About page** (<https://renderhelloapp.onrender.com/about>)

These screenshots are useful for your experiment report.

Step 7: Test the Application

Test Case 1: Home Page

Open:

<https://renderhelloapp.onrender.com>

Expected Output:

Hello! Welcome to Render PaaS.

✓ **Result: Pass**

Test Case 2: About Page

Open:

<https://renderhelloapp.onrender.com/about>

Expected Output:

This application is deployed on Render.

✓ **Result: Pass**

Conclusion

The experiment successfully demonstrated the deployment of a Flask web application using Render, a Platform as a Service (PaaS). The application was uploaded to GitHub, deployed on Render, and made accessible through a public URL without requiring any server configuration. This experiment helped in understanding how PaaS platforms simplify application hosting by managing the infrastructure while allowing developers to focus on application development.

